```
"""
Weather App with Live OpenWeather API
A beginner-friendly CLI program that
fetches real-time weather data:
  - Current temperature, feels-like,
humidity, wind, and conditions
  - 5-day forecast (daily min/max
temperature and conditions)
  - Celsius or Fahrenheit units
  - Error handling for bad city
names, invalid API keys, and network
problems

Setup:
  1. Install the library:   pip
install requests
  2. Get a free API key at:
https://openweathermap.org/api
  3. Set it as an environment
variable (recommended):
        Windows (CMD):   set
OPENWEATHER_API_KEY=your_key_here
        Mac/Linux:       export
OPENWEATHER_API_KEY=your_key_here
     Or just paste the key when the
program asks for it.
```

```
  Note: a new API key can take up to
1-2 hours to start working.
"""

import os
from collections import Counter,
defaultdict
from datetime import datetime

import requests

BASE_URL =
"https://api.openweathermap.org/data/
2.5"


# ---------- Helpers ----------
def get_api_key():
    """Use the environment variable
if set, otherwise ask the user."""
    key =
os.environ.get("OPENWEATHER_API_KEY",
"").strip()
    while not key:
        key = input("Enter your
OpenWeather API key: ").strip()
        if not key:
            print("The API key cannot
be empty.")
```

```
    return key


def get_choice(prompt,
valid_choices):
    while True:
        choice =
input(prompt).strip()
        if choice in valid_choices:
            return choice
        print(f"Invalid choice.
Please pick one of: {',
'.join(valid_choices)}")


def get_city():
    while True:
        city = input("Enter a city
name (example: Bengaluru or
London,GB): ").strip()
        if city:
            return city
        print("City name cannot be
empty.")


def unit_symbols(units):
    """Return the temperature and
wind-speed labels for the chosen
```

```
units."""
    if units == "metric":
        return "°C", "m/s"
    return "°F", "mph"


# ---------- API calls ----------
def fetch_weather(endpoint, city,
api_key, units):
    """Call the OpenWeather API and
return the JSON data (or None on
error)."""
    params = {"q": city, "appid":
api_key, "units": units}
    try:
        response = requests.get(f"
{BASE_URL}/{endpoint}",
params=params, timeout=10)
    except
requests.exceptions.ConnectionError:
        print("Connection error.
Please check your internet
connection.")
        return None
    except
requests.exceptions.Timeout:
        print("The request timed out.
Please try again.")
        return None
```

```
    except
requests.exceptions.RequestException
as error:
        print(f"Something went wrong:
{error}")
        return None

    if response.status_code == 200:
        return response.json()
    if response.status_code == 401:
        print("Invalid API key. Check
your key (new keys can take 1-2 hours
to activate).")
    elif response.status_code == 404:
        print(f"City '{city}' was not
found. Check the spelling and try
again.")
    elif response.status_code == 429:
        print("Too many requests.
Please wait a minute and try again.")
    else:
        print(f"Unexpected error
(status code
{response.status_code}).")
    return None


# ---------- Display ----------
def show_current_weather(data,
```

```
units):
    temp_unit, speed_unit =
unit_symbols(units)
    main = data["main"]
    weather = data["weather"][0]
    wind = data.get("wind", {})

    print(f"\n--- Current Weather in
{data['name']}, {data['sys']
['country']} ---")
    print(f"Conditions  :
{weather['description'].title()}")
    print(f"Temperature :
{main['temp']:.1f}{temp_unit} (feels
like {main['feels_like']:.1f}
{temp_unit})")
    print(f"Min / Max   :
{main['temp_min']:.1f}{temp_unit} /
{main['temp_max']:.1f}{temp_unit}")
    print(f"Humidity    :
{main['humidity']}%")
    print(f"Pressure    :
{main['pressure']} hPa")
    print(f"Wind        :
{wind.get('speed', 0)} {speed_unit}")


def show_forecast(data, units):
    """The API gives a forecast every
```

```
3 hours, so group it into days."""
    temp_unit, _ =
unit_symbols(units)
    days = defaultdict(list)
    for entry in data["list"]:
        date =
entry["dt_txt"].split(" ")[0]  #
"YYYY-MM-DD"
        days[date].append(entry)

    print(f"\n--- 5-Day Forecast for
{data['city']['name']}, {data['city']
['country']} ---")
    print(f"{'Date':<18}{'Min':>8}
{'Max':>8}{'Humidity':>10}
Conditions")
    print("-" * 66)

    for date in list(days)[:5]:
        entries = days[date]
        temps = [e["main"]["temp"]
for e in entries]
        humidity = sum(e["main"]
["humidity"] for e in entries) /
len(entries)
        conditions =
Counter(e["weather"][0]
["description"] for e in entries)
        most_common =
```

```
conditions.most_common(1)[0]
[0].title()
        label =
datetime.strptime(date, "%Y-%m-
%d").strftime("%a, %d %b")
        print(f"{label:<18}
{min(temps):>7.1f}{temp_unit[-1]}
{max(temps):>7.1f}{temp_unit[-1]}"
              f"{humidity:>9.0f}%
{most_common}")


# ---------- Main menu ----------
def main():
    print("=== Weather App
(OpenWeather API) ===")
    api_key = get_api_key()
    units = "metric"

    while True:
        temp_unit, _ =
unit_symbols(units)
        print(f"\nMain Menu (units:
{temp_unit})")
        print("1. Current weather")
        print("2. 5-day forecast")
        print("3. Both")
        print("4. Switch units
(Celsius / Fahrenheit)")
```

```
        print("5. Exit")
        choice = get_choice("Choose
an option (1-5): ", ["1", "2", "3",
"4", "5"])

        if choice == "4":
            units = "imperial" if
units == "metric" else "metric"
            print("Units switched
to", "Fahrenheit" if units ==
"imperial" else "Celsius")
        elif choice == "5":
            print("Goodbye!")
            break
        else:
            city = get_city()
            if choice in ("1", "3"):
                current =
fetch_weather("weather", city,
api_key, units)
                if current:

show_current_weather(current, units)
            if choice in ("2", "3"):
                forecast =
fetch_weather("forecast", city,
api_key, units)
                if forecast:
```

```
show_forecast(forecast, units)


if __name__ == "__main__":
    main()
```